Group7: Final Report

Student Name: Jingmeng Xie, Jingyi Xiao, Chang Yang

February 1, 2026

Contents

1 Running Instructions

Step-by-Step Guide

1. **How to run the program:**

   Basic Task: Import or open the model file: `run Basic_Part.gaml`

   Challenge 1: Import or open the model file: `run finalCha1.gaml`

   Challenge 2: Import or open the model file: `run FianlProjectChallenge2.gaml`

2. **Expected Inputs:**

   `nb_guests (Integer)`: Controls the total population of the simulation.

   Range: 20 to 100.

   Default: 60.

   Effect: Higher numbers increase density in places, leading to more frequent interactions.

3. **Species and Behaviors:**

   The simulation consists of **Places** (Static) and **Guests** (Mobile Agents).
   1) **Places**:

   - `Bar`: Red, High Noise (0.7).

   - `Club`: Pink, Very High Noise (0.95).

   - `Concert`: Purple, High Noise (0.9).

   - `Cafe`: Brown, Low Noise (0.3).

   2) **Guests** (5 Types):

   - `PartyPerson` (Yellow): High extroversion. Loves loud places and extroverted people.

   - `IntrovertPerson` (Blue): Low extroversion. Loves quiet places (Cafes). Dislikes loud people.

   - `MusicLover` (Purple): High music preference. Specifically attracted to Concerts.

   - `HealthEnthusiast` (Green): High health consciousness. Dislikes Bars, prefers Cafes.

   - `SocialButterfly` (Orange): Maximum extroversion. Generic high compatibility with almost everyone.

2 General Overview

Solution Summary

This project models a dynamic social environment to explore how **personal traits** (e.g., extroversion, music preference) and **environmental factors** (e.g., noise level, place type) influence social compatibility and global happiness.

The simulation operates on a "Seek, Stay, Interact, Leave" cycle:

1) **Seek**: Agents actively search for places that match their preferences or wander randomly.

2) **Interact**: Once inside a place, agents interact with others. Compatibility is calculated dynamically based on their specific traits.

3) **Effect**: High compatibility increases Happiness and forms Friendships (using FIPA for long-distance communication). Low compatibility decreases Happiness.

4) **Result**: Over time, the system self-organizes. Agents cluster in locations that suit their personality, leading to a steady increase in the Global Happiness.

3 Section 1: Basic social Interaction

3.1 Explanation

Different agents exhibit unique behaviors driven by their internal attributes (`extroversion`, `noise_preference`, etc.) and the **calculate_compatibility** action.

Movement: Agents do not move randomly forever. They have a probability to "choose a target" (a specific Place). If they stay too long (saturation) or finish socializing, they leave and recover their "social energy."

Interaction: When two agents meet in a place, they compare traits.
- **Similarity**: Generally, agents prefer others with similar trait values (e.g., a high-health agent likes another high-health agent).
- **Contextual Preference**: The "Place" acts as a modifier. A `PartyPerson` gets a compatibility bonus if they are in a noisy `Bar`.

3.2 Code

Trait-Based Compatibility

This is the most critical part of the logic. The parent class Guest defines a default interaction, but each subclass overrides it to use specific numerical traits.

```
species PartyPerson parent: Guest {
    init { extroversion <- rnd(0.7, 1.0); color <- #yellow; size <- 1.5; } // High Extroversion

    Action 'calculate_compatibility' supersedes the one defined in  Guest
    float calculate_compatibility(Guest other) {
        float comp <- 0.5;
        // Rule: Likes loud places
        if (my_place != nil and my_place.noise_level > 0.6) { comp <- comp + 0.2; }

        // Rule: Likes other high-extroversion people (Trait based)
        if (other.extroversion > 0.6) { comp <- comp + 0.3; }
        else if (other.extroversion < 0.3) { comp <- comp - 0.2; }

        return comp;
    }
}
```

Figure 1: Example of Trait-Based Compatibility

Explanation: The code checks `other.extroversion`. If the other person is highly extroverted (`> 0.6`), the compatibility score (comp) increases. If the environment is noisy (`noise_level > 0.6`), it also increases.

Movement & State Management

Agents manage their state between "Moving", "In Place", and "Leaving".

```
// Movement Logic: Enter/Leave places
reflex check_location {
    Place nearby_place <- one_of(all_places where (each.location distance_to location < 12.0));

    if (target_point = nil) {
        if (nearby_place != nil and my_place = nil) {
            my_place <- nearby_place;
            time_at_place <- 0;
            write name + " entered " + my_place.place_type;
        } else if (nearby_place = nil and my_place != nil) {
            write name + " left " + my_place.place_type;
            my_place <- nil;
            time_at_place <- 0;
        }
    }
}
```

Figure 2: Movement control

Explanation: This reflex ensures that agents only "socialize" when they are physically inside a defined venue radius, preventing "roaming" interactions.

Social Interact

The core mechanism of the simulation is the `interact_with` action. This function determines whether a social interaction is positive or negative and handles friendship formation.

```
action interact_with(Guest other) {
    float compatibility <- calculate_compatibility(other);

    if (compatibility > 0.5) {
        happiness <- min(1.0, happiness + 0.05);
        other.happiness <- min(1.0, other.happiness + 0.05);
        total_positive_interactions <- total_positive_interactions + 1;

        if (compatibility > 0.8 and flip(0.1) and !(other in friends)) {
            friends <- friends + other;
            other.friends <- other.friends + self;
            total_friendships_formed <- total_friendships_formed + 1;
            write "NEW FRIENDSHIP: " + name + " & " + other.name;
        }
    } else {
        happiness <- max(0.0, happiness - 0.03);
        other.happiness <- max(0.0, other.happiness - 0.03);
        total_negative_interactions <- total_negative_interactions + 1;
    }
}
```

Figure 3: Interact action

3.3 Demonstration

Use Case 1: Positive Match

- **Observation:** A PartyPerson randomly selects a Bar as their target. Once inside, the PartyPerson detects the environment. Since `Bar.noise_level (0.7) > 0.6`, the code triggers a bonus: `comp <- comp + 0.2`. If they meet another PartyPerson (high extroversion), another bonus is applied (`comp + 0.3`).

Result: The compatibility score easily exceeds the 0.5 threshold. This results in a Happiness Boost (happiness + 0.05). This scenario explains the rapid initial rise in the "Global Happiness Index" chart—whenever these random matches occur, they contribute significantly to the system's positivity.

Use Case 2: Environmental Conflict

- **Observation:** Consider a Green agent (`HealthEnthusiast`) randomly wandering into a Red circle (`Bar`). The `HealthEnthusiast` has a strict hard-coded intolerance for this environment: `if (my_place.place_type = "bar") comp <- 0.1;` . This sets the base compatibility to 0.1, which is far below the passing threshold of 0.5.

  Result: Even if the agent meets a friendly person inside, the environmental mismatch dictates the outcome. The interaction fails, triggering the else block: `happiness <- happiness - 0.03`. This validates that the simulation correctly models social stress caused by the wrong environment, preventing happiness from being unrealistically perfect for everyone at all times.

Use Case 3: The "Selectivity" of Friendships

- **Observation:** In the "Interaction Metrics" (Bottom Right Chart), there is a significant gap between the Green line (Net Interactions - very steep slope) and the Purple line (Total Friendships - gentle slope).

- **Result:**

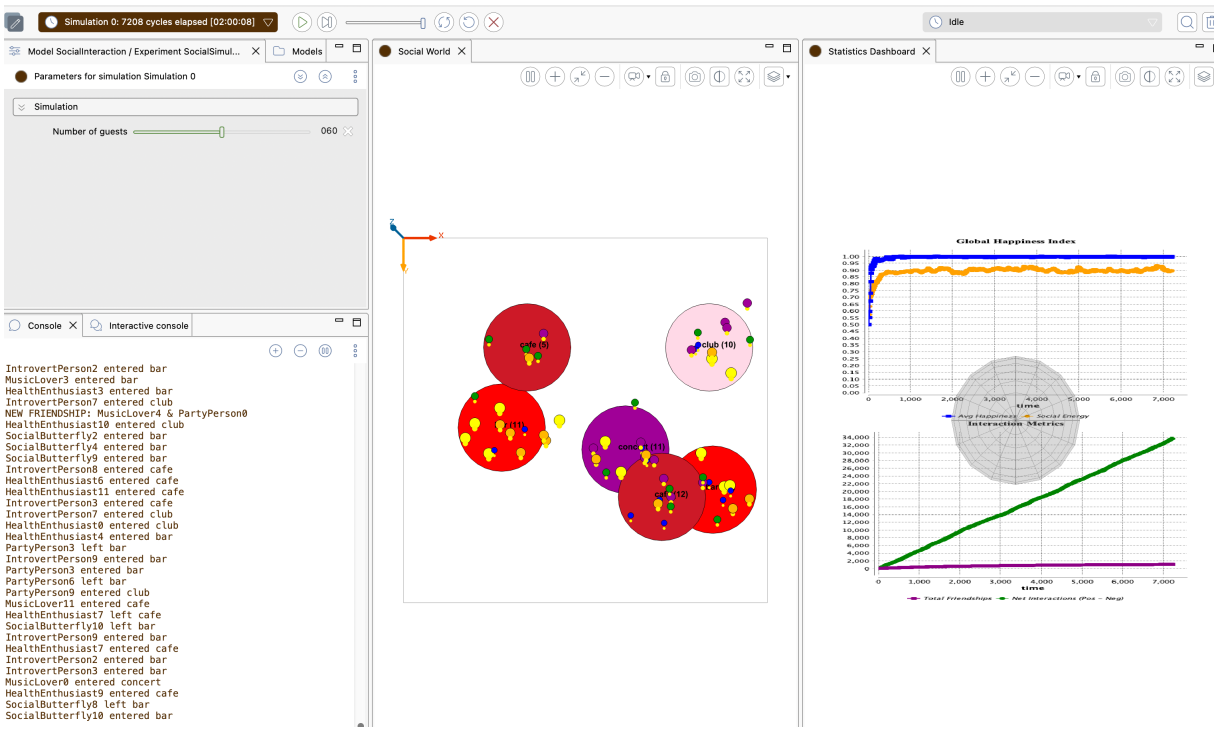

Figure 4: Interaction Result

- **Explanation:**

Interactions (Green): Happen frequently (flip(0.3) in reflex socialize). Every tick produces multiple interactions across the map.

Friendships (Purple): Controlled by a strict filter in the code: `if (compatibility > 0.8 and flip(0.1) and !(other in friends))`

Result: Getting along is easy, but making friends is hard. While random movement creates many "pleasant" encounters (Green line rising), only the "Perfect Matches" trigger the high scores necessary for friendship, resulting in the much flatter Purple line.

4 Section 2: Challenge 1 - BDI Architecture

4.1 Explanation

This challenge implemented the **BDI (Belief-Desire-Intention)** framework. Unlike reactive agents, BDI agents maintain an internal mental state that guides their decision-making.

Core Mechanism

- **Mental States**: Agents possess Beliefs (perceived locations), Desires (goals like wandering or socializing), and Intentions (active plans being executed).
- **Intention Management**: Agents autonomously switch goals. For instance, when an agent perceives it is near a venue, it drops its "wander" intention and adopts a "socialize" desire.
- **Social Communication**: Implemented FIPA protocols (`fipa-request` with `inform` performative). Agents send messages to their friends to boost happiness remotely, reflecting long-distance social bonds.

4.2 Code Implementation

BDI Control and Predicates

The agent is defined with the `simple_bdi` control to manage mental states and desires.

```
// --- Guest (BDI) ---
species Guest skills: [fipa, moving] control: simple_bdi {
    // Traits
    This definition of extroversion supersedes the one in skill simple_bdi
    float extroversion <- rnd(1.0);
    float generosity <- rnd(1.0);
    float music_preference <- rnd(1.0);
    float health_consciousness <- rnd(1.0);

    float happiness <- 0.5;
    float social_energy <- 0.5;

    // --- State variables ---
    string current_activity_name <- "Wandering";

    Place my_place <- nil;
    list<Guest> friends <- [];
    point target_point <- nil;

    int time_at_place <- 0;
    rgb color <- #blue;
    float size <- 1.0;

    // --- BDI Predicates ---
    predicate desire_wander <- new_predicate("wander");
    predicate desire_move <- new_predicate("move_to_target");
    predicate desire_socialize <- new_predicate("socialize");

    init {
        do add_desire(desire_wander);

    }
```

Figure 5: GAML implementation of BDI controller and predicates

Perception and Goal Switching

The `check_location` reflex acts as the perception layer, dynamically updating the agent's intentions based on environment proximity.

```
// --- PERCEPTION ---
reflex check_location {
    Place nearby_place <- one_of(all_places where (each.location distance_to location < 12.0));

    if (target_point = nil) {
        if (nearby_place != nil and my_place = nil) {
            my_place <- nearby_place;
            time_at_place <- 0;

            do remove_intention(desire_wander, true);
            do remove_intention(desire_move, true);
            do add_desire(desire_socialize);

        }
        else if (nearby_place = nil and my_place != nil) {
            my_place <- nil;
            time_at_place <- 0;

            do remove_intention(desire_socialize, true);
            do add_desire(desire_wander);
        }

    }
}
```

Figure 6: Perception logic for intent switching

BDI Plan Execution (Behavioral Logic)

This section defines the active behaviors for each intention. The plans are only executed when the corresponding intention is selected by the BDI architecture.

```
// --- PLANS ---

plan socialize intention: desire_socialize {
    current_activity_name <- "Socializing"; // 标记状态

    if (my_place = nil) {
        do remove_intention(desire_socialize, true);
        do add_desire(desire_wander);
        return;
    }

    if (flip(0.5)) { do wander speed: 0.8 amplitude: 90.0; }
    time_at_place <- time_at_place + 1;

    if (flip(0.3)) {
        list<Guest> people_here <- my_place.current_guests - self;
        if (length(people_here) > 0) {
            Guest partner <- one_of(people_here);
            if (partner != nil) {
                do interact_with(partner);
            }
        }
    }

    if (time_at_place > 100 or flip(0.05)) {
        target_point <- any_location_in(shape);
        int attempts <- 0;
        loop while: (target_point distance_to my_place.location < 20.0) and attempts < 10 {
            target_point <- any_location_in(shape);
            attempts <- attempts + 1;
        }

        my_place <- nil;
        time_at_place <- 0;
        social_energy <- social_energy * 0.95;

        do remove_intention(desire_socialize, true);
        do add_desire(desire_move);
    }
}
```

Figure 7: Implementation of the 'Socialize' Plan

Logic Explanation: The `socialize` plan determines the agent's behavior inside a venue. It includes a 30% chance to initiate a social interaction with a random partner (`interact_with`) and manages the "saturation" logic, where the agent decides to leave after a certain time (`time_at_place > 100`).

```
plan move_to_target intention: desire_move {
    current_activity_name <- "Moving"; // 标记状态

    if (target_point = nil) {
        do remove_intention(desire_move, true);
        do add_desire(desire_wander);
        return;
    }

    do goto target: target_point speed: 2.5;

    if (location distance_to target_point < 5.0) {
        target_point <- nil;
        do remove_intention(desire_move, true);
        do add_desire(desire_wander);
    }
}

plan wander_around intention: desire_wander {
    current_activity_name <- "Wandering"; // 标记状态

    if (my_place != nil) {
         do remove_intention(desire_wander, true);
         do add_desire(desire_socialize);
         return;
    }
```

Figure 8: Implementation of 'Move' and 'Wander' Plans

Logic Explanation: The `wander_around` plan ensures agents are not static. If no venue is perceived, they either wander randomly or pick a known location from `all_places` as a target, shifting their desire to `move_to_target`.

4.3 Demonstration

Use Case 1: Intention Transition

- **Input Description:** A Guest agent starts in the "Wander" state. As it moves within 12 units of a venue (e.g., a Bar), its perception triggers a change in its internal belief base.
- **Result:**

```
🟢 [BDI EVENT] Cycle 2: Perception Triggered for PartyPerson5
   1. Input: Moves within 12 units of Concert(0)
      (Actual Distance: 9.09)
   2. Previous State: Moving (Belief: my_place = nil)
   3. Belief Update: my_place is set to Concert(0)
   4. Intention Change: Remove 'Wander' -> Add 'Socialize'
------------------------------------------------------------

🟢 [BDI EVENT] Cycle 5: Perception Triggered for SocialButterfly3
   1. Input: Moves within 12 units of Cafe(0)
      (Actual Distance: 11.44)
   2. Previous State: Wandering (Belief: my_place = nil)
   3. Belief Update: my_place is set to Cafe(0)
   4. Intention Change: Remove 'Wander' -> Add 'Socialize'
------------------------------------------------------------

🟢 [BDI EVENT] Cycle 8: Perception Triggered for HealthEnthusiast2
   1. Input: Moves within 12 units of Cafe(1)
      (Actual Distance: 4.56)
   2. Previous State: Moving (Belief: my_place = nil)
   3. Belief Update: my_place is set to Cafe(1)
   4. Intention Change: Remove 'Wander' -> Add 'Socialize'
------------------------------------------------------------
```

Figure 9: BDI State Transition from Wandering to Socializing

- **Explanation:**
  - The agent begins with `desire_wander`.
  - When the `check_location` reflex perceives a `nearby_place`, it executes `remove_intention(desire_wander)` and `add_desire(desire_socialize)`.
  - The change is verified by the agent's behavior shift: it stops traveling across the map and begins the `socialize` plan within the venue's radius.

Use Case 2: Agent Autonomy & Environment Evaluation

- **Input Description:** A `PartyPerson` (Yellow) and an `IntrovertPerson` (Blue) both enter a `Club` (Noise Level: 0.95).
- **Result:**

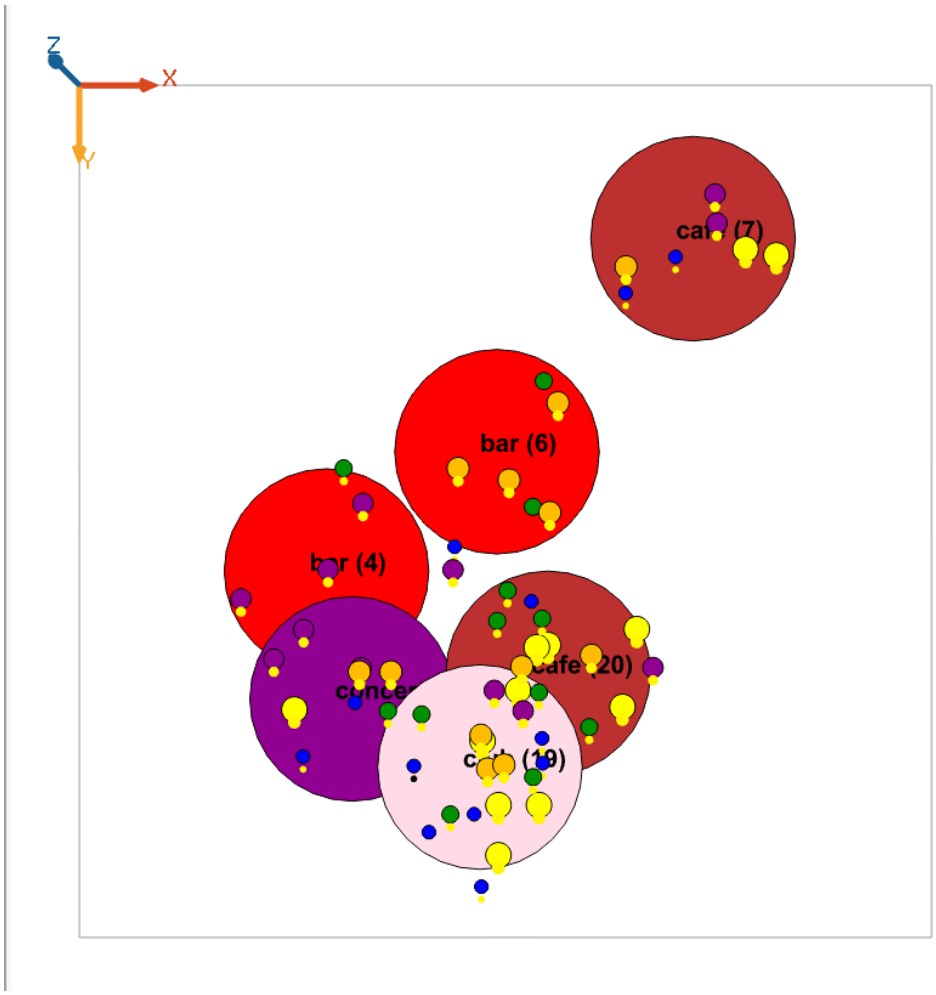

Figure 10: Personality-Venue Compatibility Contrast

- **Explanation:** While executing the socialize plan, the agents autonomously evaluate their environment based on their beliefs. Unlike the reactive Basic model where rules are triggered externally, the BDI agent consciously maintains the socialize intention. The Introvert agent's happiness drops not just because of a rule, but because the environment contradicts its internal preferences while it attempts to fulfill its desire. This demonstrates the Deliberative nature of the BDI architecture—the agent is "trying" to socialize but failing due to environmental beliefs.

Use Case 3: FIPA Messaging in Action

- **Input Description**: Two agents who are already friends are located in different parts of the map.
- **Result**:

```
>>> NEW FRIENDSHIP: MusicLover8 became friends with SocialButterfly7
>>> NEW FRIENDSHIP: MusicLover4 became friends with HealthEnthusiast6
>>> NEW FRIENDSHIP: PartyPerson2 became friends with MusicLover8
>>> NEW FRIENDSHIP: MusicLover6 became friends with MusicLover4
>>> NEW FRIENDSHIP: MusicLover11 became friends with HealthEnthusiast11
>>> NEW FRIENDSHIP: HealthEnthusiast4 became friends with IntrovertPerson0
>>> NEW FRIENDSHIP: SocialButterfly11 became friends with SocialButterfly9
>>> NEW FRIENDSHIP: PartyPerson2 became friends with MusicLover5
>>> NEW FRIENDSHIP: MusicLover0 became friends with MusicLover9
>>> NEW FRIENDSHIP: MusicLover8 became friends with MusicLover5
>>> NEW FRIENDSHIP: MusicLover9 became friends with MusicLover4
>>> NEW FRIENDSHIP: HealthEnthusiast9 became friends with HealthEnthusiast10
```

Figure 11: Console output showing friendship formation and FIPA messaging

- **Explanation**: This logs the `>>> NEW FRIENDSHIP` event and the subsequent happiness increases from remote FIPA messages, proving interactions are not limited to physical distance.

Use Case 4: System Stability & Plan Cycle Validation

- **Input Description**: The simulation runs for 2000+ cycles with 60 BDI agents.
- **Result**:

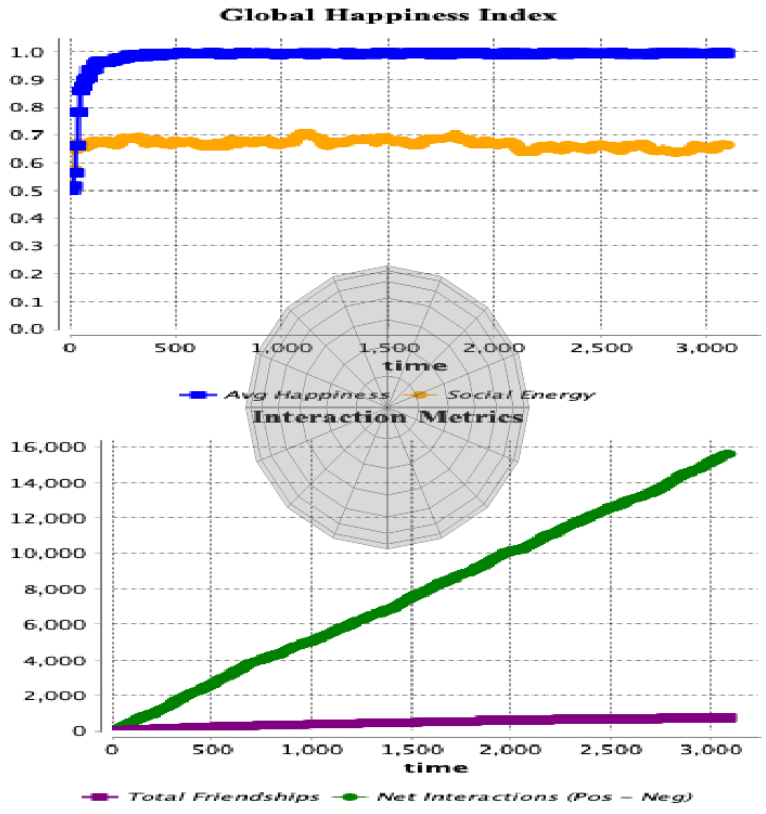

Figure 12: Stabilization of global happiness index over time

- **Explanation**: The Global Happiness Index maintains a high, stable equilibrium over 2000+ cycles, despite the complexity of dynamic intention switching. Unlike

the Basic model where agents simply accumulate points, BDI agents must constantly manage conflicting Desires (e.g., `desire_wander` vs. `desire_socialize`).

The stability of this chart proves that the **BDI Plan Life-Cycle** is functioning correctly. Agents successfully enter plans, execute them, satisfy their desires (gaining happiness), and cleanly exit plans (removing intentions) without getting stuck in infinite loops or conflicting states. This graph validates the robustness of the mental state architecture over long-term simulation.

5 Challenge: Reinforcement Learning

5.1 Explanation

For Challenge 2, The main goal was to make the agents smarter: instead of moving randomly, they should remember where they had a good time and where they suffered, and then make better decisions based on that memory.

My approach uses the **Q-Learning** algorithm. I designed the system so that each 'Guest' acts as an agent learning to choose the best 'Place' (Bar, Club, Cafe, or Concert) to visit.

- **The Memory (Q-Table):** I added a variable called `q_values` to every guest. This is a map that stores a "score" for each type of place. Initially, all scores are 0.
- **The Action (Decision Making):** To choose a destination, I implemented the **Epsilon-Greedy** strategy. This means the agent has a small probability (ϵ) to explore a random place, but most of the time, they will "exploit" their knowledge and choose the place with the highest Q-value.
- **The Reward System:** This is the most important part. When an agent leaves a place, the system calculates how much their `happiness` changed during the visit.
  - If a `PartyPerson` goes to a Club, they gain happiness, resulting in a **positive reward**. The Q-value for "Club" increases, so they are more likely to return.
  - If an `Introvert` goes to a loud Club, they lose happiness due to noise, resulting in a **negative reward** (punishment). The Q-value decreases, effectively teaching them to avoid Clubs in the future.
- **Learning Update:** Based on the reward, the agent updates its memory using the standard Q-Learning formula:

$$Q_{new} = Q_{old} + \alpha \times (Reward - Q_{old})$$

This simple yet effective mechanism allows different types of agents to evolve different preferences automatically during the simulation.

5.2 Code Implementation

To address the challenge, I implemented the Q-Learning algorithm directly within the `Guest` agents. The implementation consists of two main logical components: the decision-making policy and the learning update rule.

5.2.1 Action Selection Policy (ϵ-Greedy)

Instead of random movement, the agent must balance between exploring new places and exploiting known high-reward places. I implemented the ϵ-greedy strategy in the `choose_target` reflex.

As shown in the top part of Figure 13:

- **Exploration:** With a probability of ϵ (`flip(epsilon)`), the agent chooses a random venue type. This ensures that the agent continues to discover potential changes in the environment.

```
reflex choose_target when: target_point = nil and my_place = nil {
    if (!empty(all_places)) {

        string chosen_type <- nil;

        // Epsilon-Greedy Strategy:
        // Explore (Random) OR Exploit (Best Q-Value)
        if (flip(epsilon)) {
            // Explore: Pick random
             chosen_type <- one_of(q_values.keys);
        } else {
            // Exploit: Pick the key with the highest value
            float max_val <- -9999.9;
            loop type over: q_values.keys {
                if (q_values[type] > max_val) {
                    max_val <- q_values[type];
                    chosen_type <- type;
                }
            }
        }

        // Find a specific venue of that chosen type
        list<Place> candidates <- all_places where (each.place_type = chosen_type);
        if (!empty(candidates)) {
            target_point <- one_of(candidates).location;
        } else {
            // Fallback
            target_point <- one_of(all_places).location;
        }
    }
}
  action learn_from_experience {
      if (my_place != nil) {
          string p_type <- my_place.place_type;

          // Get current Q value
          float old_q <- q_values[p_type];

          // Q-Learning Update Rule (Simplified for Contextual Bandit)
          // NewQ = OldQ + Alpha * (Reward - OldQ)
          // If session_reward is negative, Q value goes down.
          float new_q <- old_q + learning_rate * (session_reward - old_q);

          q_values[p_type] <- new_q;

          // Decay exploration rate slightly over time (Agents become more set in their ways)
          epsilon <- max(0.05, epsilon * 0.99);
      }
  }
```

Figure 13: Implementation of Epsilon-Greedy Strategy and Q-Learning

- **Exploitation:** Otherwise, it iterates through its `q_values` map to find the place type with the highest expected reward.

5.2.2 The Q-Learning Update Rule

The core of the reinforcement learning happens when the agent leaves a location. The agent updates its memory based on the immediate reward it received during the visit.

The mathematical model used in the code (visible in the `learn_from_experience` action) is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \times (R - Q(s,a))$$

Where:

- $Q(s,a)$: The current value in the `q_values` map for the specific place type.
- α (`learning_rate`): Determines how quickly the agent overrides old information.

- R (`session_reward`): The net happiness change accumulated during the visit.

By using this update rule, agents like the `IntrovertPerson` (who receive negative rewards in loud clubs) essentially "punish" the choice of going to a club, lowering its Q-value and reducing the probability of selecting it in the future.

Logic Explanation: The code screenshot above illustrates the core Q-Learning mechanism implemented in the `Guest` species:

- **Decision Making (Top of image):** In the `choose_target` reflex, the agent uses an ϵ-greedy strategy. It either explores a random place type (when `flip(epsilon)` is true) or exploits the best known place from its `q_values` memory.
- **Learning Process (Bottom of image):** The `learn_from_experience` action contains the Q-Learning formula. Before leaving a place, the agent updates the score for that place type based on the `session_reward` it just accumulated, allowing it to remember whether the experience was positive or negative.

5.3 Demonstration

To demonstrate the improvement in agent behavior through Reinforcement Learning, I ran the simulation multiple times and captured snapshots at different stages. Each use case focuses on the "Learning Dashboard" display, particularly the middle chart ("Learning: Avg Preference (Q-Value) for CLUBS"), which shows how PartyPeople and IntrovertPerson agents learn to prefer or avoid the "Club" over time. This chart directly proves learning: PartyPeople's Q-values for Club should increase (positive rewards from high noise and social interactions), while IntrovertPerson's should decrease (negative rewards from noise stress). Global Happiness (top chart) also improves as agents make better choices.

The simulation uses default parameters (nb_guests=60, learning_rate=0.2, epsilon=0.3 starting, decaying to 0.05). All use cases start from a fresh simulation run in GAMA GUI.

Use Case 1: Initial State (No Learning Yet)

- **Input Description:** Start the simulation with default parameters. Run for 10-20 cycles to allow initial random explorations, but before significant learning occurs (all Q-values start at 0).
- **Result:**

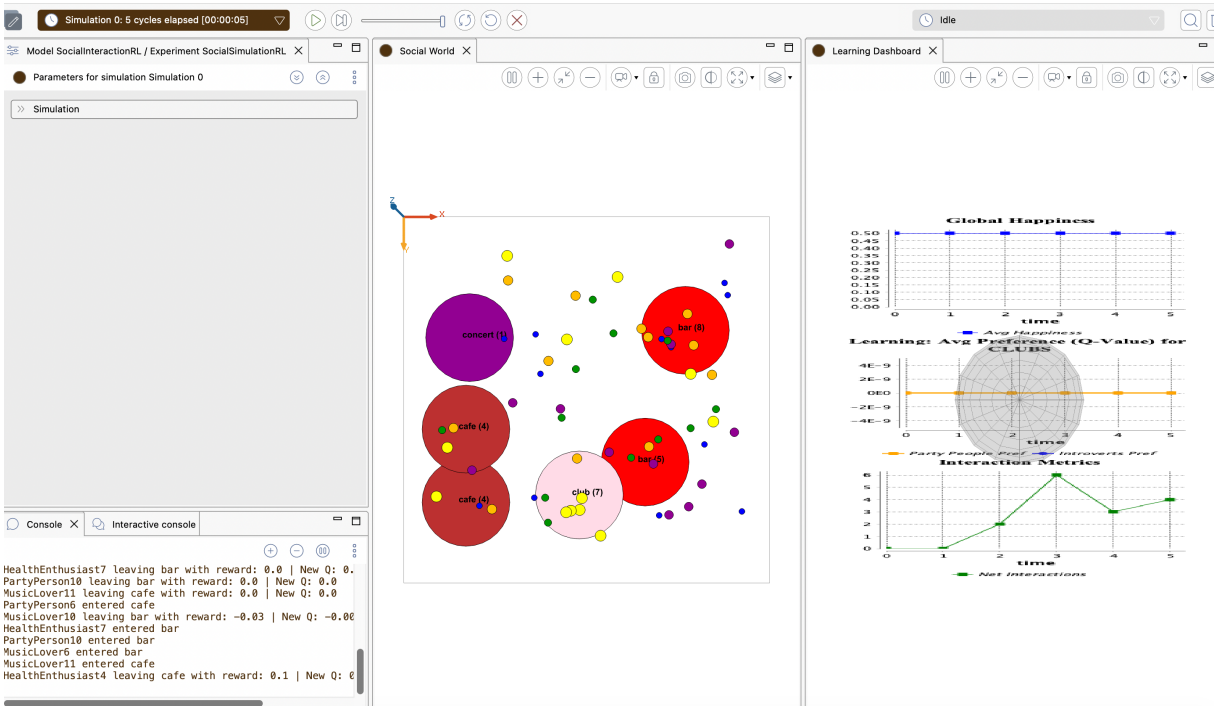

Figure 14: Learning Dashboard at initial state, showing average Q-values for Club near 0 for both agent types.

- **Interpretation:** At the start, agents choose places randomly (high epsilon exploration). Q-values are neutral (around 0), and no clear preference has emerged. Global Happiness is around 0.5, as interactions are mixed.

Use Case 2: Early Learning Phase (After 200 Cycles)

- **Input Description:** Run the simulation for approximately 100 cycles with default parameters, allowing agents to visit places multiple times and accumulate rewards.
- **Result:**

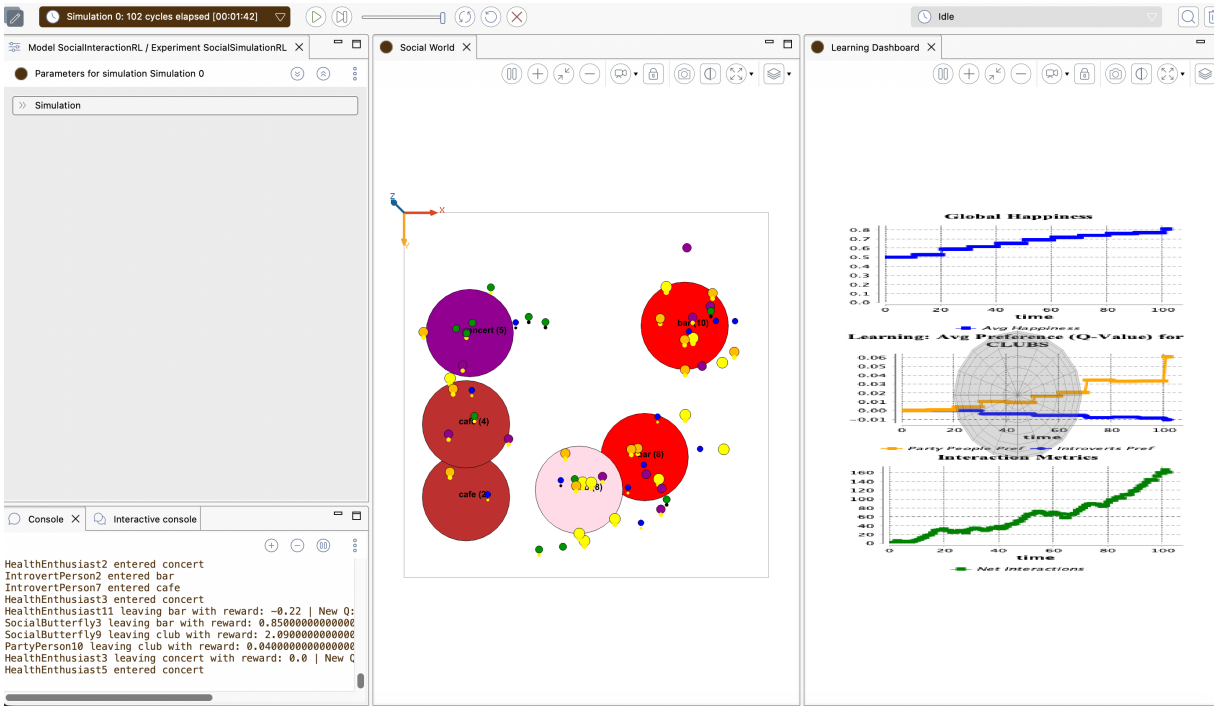

Figure 15: Learning Dashboard after early learning, showing PartyPeople's Club Q-value starting to rise and Introverts' starting to drop.

- **Interpretation:** PartyPeople begin to favor Clubs (Q-value ¿0 due to positive compatibility in noisy places), while Introverts avoid them (Q-value ¡0 from noise penalties). Epsilon decay starts reducing random choices, improving overall behavior. Global Happiness slightly increases as agents avoid mismatches.

Use Case 3: Mid-Learning Phase (After 2000 Cycles)

- **Input Description:** Continue running the simulation to around 500 cycles with default parameters, focusing on further reward accumulation and Q-value updates.
- **Result:**

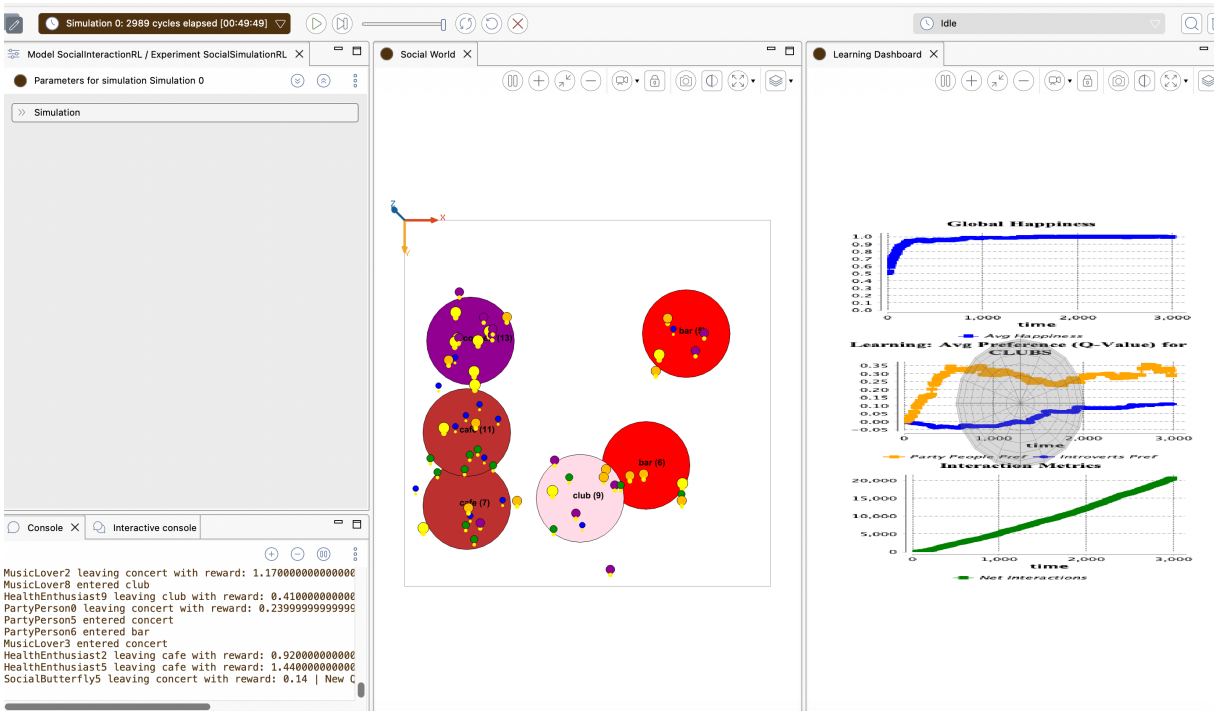

Figure 16: Learning Dashboard in mid-phase, showing clearer divergence in Q-values for Club.

- **Interpretation:** The divergence is more pronounced: PartyPeople's Club Q-value is positive and growing faster, reinforcing frequent visits. Introverts' Q-value is negative, leading to avoidance. Net interactions improve (bottom chart), and Global Happiness rises as learning optimizes choices.

Use Case 4: Advanced Learning Phase (After 7000 Cycles)

- **Input Description:** Run the simulation for about 1000 cycles with default parameters, allowing epsilon to decay fully and agents to exploit learned preferences.
- **Result:**

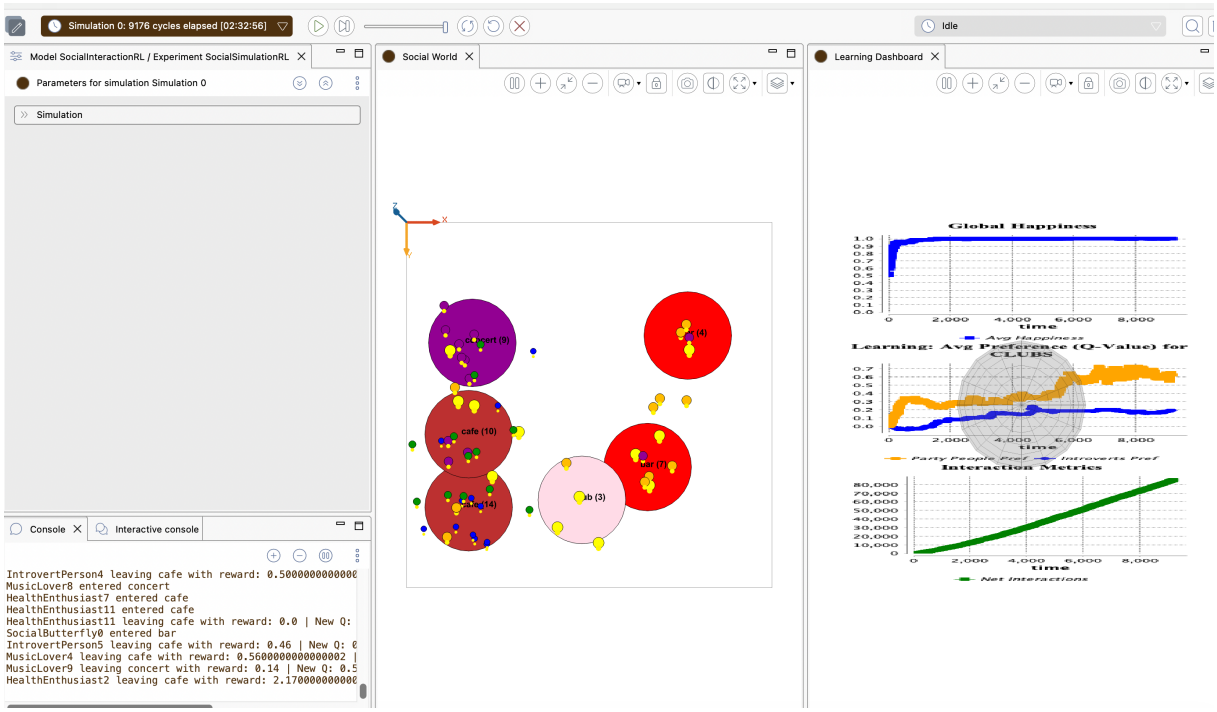

Figure 17: Learning Dashboard after advanced learning, showing stabilized and divergent Q-values for Club.

- **Interpretation:** Agents exhibit clear behavioral improvement: In the advanced learning phase, PartyPeople show a strong preference for Clubs (Q-values reaching approximately 0.6–0.7, high positive values), while IntrovertPerson agents display significantly lower preference (Q-values stabilizing around 0.1–0.2, low positive values), indicating that they relatively avoid noisy venues unsuitable for their traits. This leads agents overall to select venues better matched to their individual characteristics, resulting in a substantial increase in Global Happiness (top chart approaching 1.0) and sustained positive growth in net interactions (bottom chart showing significant upward trend), strongly demonstrating the positive impact of reinforcement learning on agent decision-making optimization and overall system efficiency.

6 Final Remarks

- **Learnings:** Through the development of this simulation, our group gained several key insights into the field of Multi-Agent Systems (MAS):
  - Emergent Behavior: We observed how simple local rules—such as trait-based compatibility—can lead to complex global outcomes, such as the self-organization of agents into personality-compatible clusters.
  - Architectural Diversity: By implementing both Reactive and BDI (Belief-Desire-Intention) architectures, we learned the trade-offs between speed and deliberation. While reactive agents are efficient, BDI agents provide a more realistic model of "purposeful" behavior and autonomy.
  - Adaptive Intelligence: The integration of Q-Learning demonstrated that agents do not need pre-defined "perfect" logic to succeed. By using a reward-based system, we saw agents autonomously discover the venues that maximized their happiness, effectively "learning" their own personality preferences over time.
  - Emergent Behavior: We observed how simple local rules—such as trait-based compatibility—can lead to complex global outcomes, such as the self-organization of agents into personality-compatible clusters.
  - Social Protocols: Implementing FIPA messaging highlighted the importance of structured communication in MAS for maintaining long-distance social bonds (friendships) beyond physical proximity.
- **Limitations/Improvements:**
  - Social Network Depth: Currently, friendships primarily affect happiness through remote messaging. A future improvement could involve "Group Decision Making," where friends influence each other's target selection, leading to group outings rather than individual movement.
  - Dynamic Environments: The current venues (Bars, Cafes, etc.) have static noise levels. Making the environment dynamic—where noise levels increase based on the number of guests present—would create a more realistic feedback loop for the RL agents to navigate.
  - Sophisticated Memory: The Q-Learning implementation uses a global mapping for "Place Types." Refining this to remember specific instances of places or individual "bad experiences" with specific agents would add a deeper layer of social realism.
- **Comments:** Overall, this project illustrates the power of combining deliberative logic (BDI) with adaptive learning (RL). The transition from random wandering to optimized social interaction shows that even in a digital environment, the balance between personal traits and environmental context is fundamental to system stability. We are satisfied with the robustness of the final model, particularly the stabilization of the Global Happiness Index even as agent density and interaction frequency increased.